

Học Sâu (Deep Learning)

Chương 18: Kỹ năng Thực chiến & Tối ưu hóa Mô hình

Đại học Đông Á

July 24, 2026

1. Tối ưu hóa Siêu tham số (Hyperparameter Tuning)
2. Suy diễn Đám đông - Model Ensembling
3. Huấn luyện Phân tán (Distributed Training)
4. Tối ưu Bộ nhớ: Mixed-Precision & Quantization
5. Tổng kết: Vòng lặp Tiến bộ

Tham số và Siêu tham số

- **Tham số (Parameters):** Là các trọng số (Weights) và độ lệch (Biases) mà mô hình tự động học được thông qua thuật toán Gradient Descent và Backpropagation.
- **Siêu tham số (Hyperparameters):** Là các thiết lập kiến trúc do con người định nghĩa trước khi huấn luyện. Ví dụ: Số lớp (layers), số node ẩn, Learning Rate, kích thước Batch, hàm Kích hoạt.
- **Vấn đề:** Không gian Siêu tham số không liên tục và không thể đạo hàm, do đó không thể dùng Gradient Descent để tối ưu.

Các chiến lược tìm kiếm Siêu tham số

- **Grid Search (Tìm kiếm dạng lưới):** Duyệt qua mọi tổ hợp có thể của các siêu tham số. Tuyệt đối chính xác nhưng tốn thời gian theo cấp số nhân (Combinatorial Explosion).
- **Random Search (Tìm kiếm ngẫu nhiên):** Chọn ngẫu nhiên các tổ hợp trong không gian định sẵn. Nhanh hơn Grid Search và thường tìm được cấu hình đủ tốt trong thời gian ngắn.
- **Bayesian Optimization (Tối ưu hóa Bayes):** Sử dụng mô hình thống kê (Surrogate Model) để học từ các lần thử nghiệm trước, từ đó dự đoán tọa độ siêu tham số nào có khả năng cho ra kết quả tốt nhất ở lần thử tiếp theo.

- Trong thực tế, Kỹ sư Machine Learning không điều chỉnh siêu tham số bằng tay. Họ sử dụng thư viện tự động như **KerasTuner**.
- Quy trình hoạt động của KerasTuner:
 - ➊ Định nghĩa Không gian tìm kiếm (Search Space) (Ví dụ: Số lớp từ 2 đến 5, Learning rate $\{0.01, 0.001\}$).
 - ➋ Xác định Hàm mục tiêu (Objective) (Ví dụ: `val_accuracy` cao nhất).
 - ➌ Chạy thuật toán dò tìm (Random Search, Hyperband, hoặc Bayesian Optimization).
 - ➍ Lấy ra mô hình tốt nhất để triển khai.

Triết lý Ensemble: Suy diễn Đám đông

- **Ensembling** (Kết hợp mô hình) là kỹ thuật gộp dự đoán của nhiều mô hình khác nhau để tạo ra một dự đoán cuối cùng mạnh mẽ hơn.
- Triết lý: "Nhiều mô hình yếu cộng lại thành một mô hình mạnh".
- Dựa trên nguyên lý Thống kê học: Khi kết hợp các mô hình, ta đang thực hiện trung bình hóa các sai số. Nếu các mô hình có lỗi độc lập với nhau, sai số tổng thể sẽ triệt tiêu lẫn nhau.

Các kỹ thuật Ensemble phổ biến

- **Averaging (Trung bình hóa):** Trộn kết quả xác suất của nhiều mô hình (Thường áp dụng cho Neural Networks).
- **Bagging (Bootstrap Aggregating):** Huấn luyện cùng một mô hình trên nhiều tập con ngẫu nhiên của dữ liệu (VD: Random Forest). Giúp giảm phương sai (Variance).
- **Boosting:** Huấn luyện các mô hình tuần tự, mô hình sau tập trung sửa lỗi sai của mô hình trước (VD: XGBoost, AdaBoost). Giúp giảm độ lệch (Bias).

Toán học của việc Trung bình hóa (Weighted Average)

- Thay vì lấy trung bình cộng đơn thuần, Kỹ sư ML thường dùng **Trung bình có trọng số**:

$$\hat{Y} = \sum_{i=1}^N w_i \cdot M_i(X)$$

Trong đó $\sum w_i = 1$ và các w_i được tối ưu hóa dựa trên độ chính xác của từng mô hình M_i .

- Bất đẳng thức Jensen:** Nếu hàm mất mát là hàm lồi (Convex), thì sai số của mô hình Ensemble luôn nhỏ hơn hoặc bằng trung bình sai số của các mô hình thành phần:

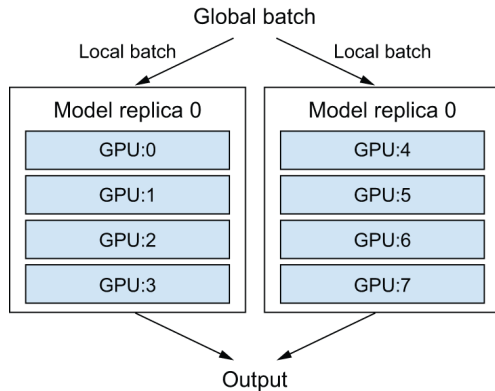
$$L(\mathbb{E}[M_i]) \leq \mathbb{E}[L(M_i)]$$

Tại sao cần Huấn luyện phân tán?

- Các mô hình SOTA (Ví dụ: LLMs, Vision Transformers) mất tới hàng chục nghìn năm để huấn luyện nếu chỉ dùng 1 GPU.
- Bằng cách nối mạng hàng nghìn GPU/TPU với nhau thành một cụm siêu máy tính (Cluster), thời gian huấn luyện giảm xuống chỉ còn vài tuần.
- Vấn đề cốt lõi: Làm sao chia việc cho hàng nghìn GPU chạy đồng thời mà không bị xung đột bộ nhớ và đảm bảo tiến trình Gradient cập nhật đúng?

Phân tán Dữ liệu vs Phân tán Mô hình

- **Data Parallelism:** Mỗi GPU chứa 1 bản sao y hệt của mô hình. Dữ liệu (Batch) được chia nhỏ ra (Ví dụ Batch 1024 chia thành 8 phần cho 8 GPU).
- **Model Parallelism:** Khi mô hình quá lớn không nhét vừa RAM 1 GPU, ta phải "chặt đứt" các Layer của mạng Nơ-ron và rải chúng ra nhiều GPU khác nhau.



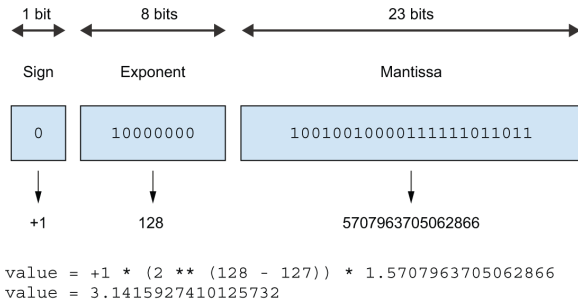
Hình 18.1: Hai mô hình tính toán phân tán

Đồng bộ hóa Gradient (All-Reduce Algorithm)

- Trong Data Parallelism, nếu mỗi GPU nhận 1 tập dữ liệu con khác nhau, đạo hàm (Gradient) của chúng tính ra sẽ khác nhau.
- Nếu cập nhật trọng số trực tiếp, 8 GPU sẽ tự tạo ra 8 mô hình khác biệt hoàn toàn!
- **Thuật toán All-Reduce:** Ở mỗi bước (Step), toàn bộ GPU phải đồng bộ hóa. Chúng truyền mạng gửi Gradient cho nhau, tính Gradient trung bình rồi cập nhật đồng loạt, đảm bảo tất cả GPU đều giữ phiên bản mô hình y hệt nhau.

Biểu diễn dấu phẩy động trong Khoa học Máy tính

- Máy tính lưu trữ số thực dưới định dạng chuẩn IEEE 754.
- Float32 (Single Precision):** Dùng 32 bits. Rất chính xác, nhưng tốn RAM và băng thông nhớ.
- Float16 (Half Precision):** Dùng 16 bits. Tiết kiệm RAM và tính toán cực nhanh trên Tensor Cores, nhưng dễ bị tràn số (Overflow/Underflow).



Hình 18.2: Độ chính xác của dấu phẩy động

Kỹ thuật Mixed-Precision Training

- **Mixed-Precision:** Kết hợp ưu điểm của cả FP32 và FP16.
- Trong quá trình Forward Pass (Tính toán) và Backward Pass (Tính Gradient), hệ thống ép kiểu dữ liệu xuống **FP16** để CPU/GPU tăng tốc độ Matrix Multiplication lên gấp nhiều lần.
- Tuy nhiên, khi cộng Gradient vào Trọng số (Weights), hệ thống dùng **FP32** làm Master Weights để tránh việc các số Gradient vi phân quá nhỏ bị làm tròn về 0 (Underflow).

Lượng tử hóa (Quantization)

- Để đưa các mô hình AI khổng lồ lên chạy trên Điện thoại hoặc IoT (Edge Devices), ta phải nén chúng lại.
- **Quantization:** Kỹ thuật ép kiểu các Trọng số từ số thực Float32 xuống số nguyên Int8. Giúp giảm 4 lần dung lượng RAM lưu trữ và chạy cực nhanh trên CPU.
- **Phương trình Lượng tử hóa tuyến tính:**

$$x_q = \text{round} \left(\frac{x}{S} \right) + Z$$

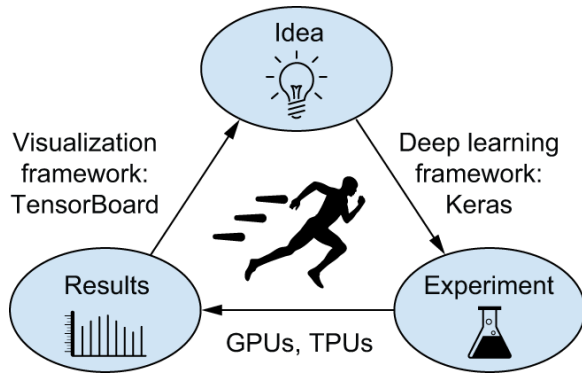
Trong đó S là hệ số Tỷ lệ (Scale) và Z là điểm bù (Zero-point).

Mất mát thông tin do Lượng tử hóa

- Khi ép từ dải vô hạn của số thực xuống chỉ còn 256 giá trị của Int8, thông tin chắc chắn bị suy giảm.
- **Post-Training Quantization (PTQ):** Train xong mô hình bằng Float32 rồi ép kiểu. Đơn giản, nhanh gọn nhưng giảm độ chính xác của mô hình.
- **Quantization-Aware Training (QAT):** Trong lúc Train Float32, hệ thống mô phỏng việc bị cắt cụt xuống Int8 ở quá trình Forward Pass. Do đó mô hình tự thích nghi với độ nhiễu lượng tử, giữ nguyên độ chính xác khi thực sự triển khai.

Vòng lặp Tiến bộ (The Loop of Progress)

- AI không bao giờ hoàn hảo ngay từ lần Train đầu tiên.
- Một kỹ sư Machine Learning phải liên tục thực thi vòng lặp: **Đánh giá (Evaluate)** → **Phân tích lỗi (Error Analysis)** → **Thu thập dữ liệu / Tinh chỉnh** → **Huấn luyện lại**.
- Đây là chìa khóa để đạt trạng thái State-of-the-Art.



Hình 18.3: Vòng lặp phát triển vô tận của ML

Tổng kết Chương 18

- **Hyperparameter Tuning:** Dùng công cụ (KerasTuner, Bayesian Optimization) thay vì thử công để dò tìm không gian kiến trúc.
- **Ensembling:** Áp dụng Bất đẳng thức Jensen và Suy diễn đám đông để đẩy hiệu suất mô hình vượt qua giới hạn của mạng đơn lẻ.
- **Hardware Optimization:** Hiểu rõ Distributed Training (Multi-GPU), Mixed Precision (FP16/FP32), và Quantization (Int8) là yếu tố quyết định để đưa AI vào cấp độ Sản xuất (Production).
- **Kết thúc giáo trình:** Các bạn đã trang bị đủ hành trang từ Lý thuyết căn bản đến Tối ưu hóa hệ thống AI ở cấp độ thực chiến.